E-Parada: Capstone Defense Presentation Script (Concise & Word-for-Word)

Title: *E-Parada: A Digital Marketplace for Parking Spaces*

Presenters: Bustria, Christian Jake C. | Gamad, Leticia C. | Ovilla, Shan Jenis C. | Valiña, Resha Lorraine P.

Estimated Total Speaking Time: ~12–15 Minutes (Concise, Direct, and 100% True to the System)

OPENING / TITLE SLIDE (~30 Seconds)

Speaker:

> "Good day to our esteemed panel members and adviser. We are here to present our capstone project titled **E-Parada: A Digital Marketplace for Parking Spaces**.
>
> In rapidly urbanizing cities like **Calamba City, Laguna**, motorists struggle to find safe, available parking, while many residential driveways and private commercial lots remain idle. **E-Parada** bridges this gap through a **100% software-driven, cross-platform digital marketplace** that connects parking space providers and drivers without requiring expensive stationary IoT hardware or physical gate barriers."

CHAPTER 1: INTRODUCTION

SLIDE 1: Objectives of the Study (~1 Minute)

Speaker:

> "Moving to **Slide 1**, our general objective is to enhance efficiency, effectiveness, and reliability in parking management by designing and developing **E-Parada**, a cross-platform digital marketplace connecting residential and commercial parking providers with motorists in Calamba City, Laguna.
>
> Specifically, our study aims to:
>
> 1. **First, design** a centralized marketplace architecture featuring parking area and **2D floor plan slot management**, **OCR-assisted vehicle registration**, location-based searching, **cryptographic QR code access workflows**, automated duration billing, and reservation-bound communication.
> 2. **Second, develop** the E-Parada ecosystem—consisting of our **Flutter Android mobile application** and **Laravel 12 administrative web portal** backed by **Supabase PostgreSQL 15+**—following the **Incremental Development Model**, while integrating **Google Maps** for navigation, **Tesseract OCR** for license plates, **WebRTC** for voice calling, **FCM** and **Brevo** for notifications, and **PayMongo API v1** for GCash payments.
> 3. **And third, test and evaluate** the system's software product quality through **IT developer evaluation** and technical benchmarking under the **ISO/IEC 25010:2011** standard across **Functional Suitability, Performance Efficiency, Usability, and Reliability**."

SLIDE 2: Scope of the Study (~1 Minute)

Speaker:

"On **Slide 2**, E-Parada operates across an **Android mobile application** and an **Administrative Web Portal**, synchronized in real time via **Supabase PostgreSQL**, targeting residential driveways, vacant lots, and commercial parking spaces in **Calamba City**.

The system serves three core roles:

- **Parking Space Providers** can verify property ownership, list parking spaces with operating schedules, configure interactive **2D floor plan slot layouts** with specific vehicle dimensions and clearances, set hourly rates within admin-approved boundaries, approve reservations, and validate entry and exit using our **QR code scanner** or manual `RES-#` **reservation code fallback**.
- **Drivers** can register up to three vehicles using **Tesseract OCR plate extraction**, search for compatible nearby slots via **Google Maps**, generate an encrypted **HMAC-SHA256 QR pass**, settle automated duration and overstay bills via **Cash or GCash through PayMongo**, and contact providers through **reservation-bound chat and WebRTC voice calls**.
- **System Administrators** govern the platform by reviewing KYC documents and OCR vehicle registrations, enforcing minimum and maximum parking rate boundaries, monitoring live QR and payment logs, mediating disputes, and exporting **CSV forensic audit reports**."

SLIDE 3: Limitations of the Study (~1 Minute)

Speaker:

"**Slide 3** outlines our operational and technical boundaries:

1. **Platform & Connectivity:** The mobile app targets **Android 8.0 and above**, and the system requires an active internet connection for real-time database synchronization, Google Maps routing, and PayMongo payment webhooks.
2. **100% Software-Driven Architecture:** E-Parada intentionally excludes physical hardware—meaning **no automated gate barriers, no RFID readers, no ultrasonic ground sensors, no predictive AI demand modeling, and no live CCTV streaming**. Instead, entry and exit checkpoints are validated directly through the parking provider's smartphone camera.
3. **Verification & Billing:** User IDs, property documents, and vehicle records undergo administrative review on the platform rather than direct government LTO/PNP database queries. Billing is strictly calculated from validated QR **time-in** and **time-out** timestamps within admin-approved rate bounds, with 100% of the parking fee going directly to the parking provider.
4. **Evaluation Scope:** Finally, rather than mass public deployment, the system is evaluated through controlled technical benchmarking and end-to-end software quality assessment by **qualified IT developers**."

SLIDE 4: Conceptual Framework — IPO Model (~1 Minute)

🎙️ **Speaker:**

> "**Slide 4** presents our Conceptual Framework using the **Input–Process–Output (IPO) Model**.
>
> - **For our Inputs:** We gathered role requirements for Drivers, Providers, and Admins; architectural designs including **2D Floor Plan wireframes** and **Crow's Foot ERDs**; and our updated development stack—**Flutter 3.x**, **PHP 8.2 / Laravel 12 on Docker and Render**, **Supabase PostgreSQL 15+**, **Google Maps**, **Tesseract OCR**, **Google ML Kit (`mobile_scanner`)**, **WebRTC**, **FCM**, **Brevo SMTP**, and **PayMongo API v1**—alongside our 20 test cases and ISO/IEC 25010 instrument.
> - **For our Process:** We executed five stages: Planning and Analysis, System Design, **Incremental Development across five sequential increments**, System Integration and Testing, and finally **System Evaluation by IT Developers**.
> - **This yields our Output: E-Parada: A Digital Marketplace for Parking Spaces**—a verified cross-platform Flutter Android application and Laravel Administrative Web Portal."

CHAPTER 2: REVIEW OF RELATED LITERATURE AND STUDIES

SLIDE 5: Literature Matrix (~20 Seconds)

🎙️ **Speaker:**

> "Moving to Chapter 2 on **Slide 5**, here is our **Literature Matrix**, which summarizes the local and foreign literature and studies that informed our architectural and algorithmic design decisions."
>
> *(Pause briefly to let the panel view the matrix table on the slide.)*

SLIDE 6: Literature & Design Synthesis (~1 Minute)

🎙️ **Speaker:**

> "**Slide 6** synthesizes our literature review across six core themes and highlights the research gap E-Parada addresses:
>
> 1. **Geolocation & Urban Navigation:** Because mobile GPS experiences multipath drift in dense areas, coupling device geolocation with the **Google Maps Platform API** ensures accurate pin discovery and turn-by-turn routing to residential parking spaces.
> 2. **Smart Parking vs. Software-Only Design:** Prior smart parking studies rely heavily on costly IoT ultrasonic sensors and microcontrollers. E-Parada closes this cost barrier by using a **100% software-driven architecture** with interactive **2D floor plan slot mapping** and smartphone camera validation.
> 3. **Digital Sharing Economy:** Our platform unlocks idle private driveways and vacant lots to increase Calamba City's parking supply without building new physical infrastructure.

4. **Cryptographic QR Validation:** To prevent screenshot sharing and replay attacks common in static QR codes, E-Parada implements **HMAC-SHA256 cryptographic signing** with strict single-use state transitions.
5. **Transactional Integrity:** Using **Supabase PostgreSQL 15+** with pessimistic row-level locking (`SELECT ... FOR UPDATE`) eliminates race-condition double bookings while streaming live slot updates via WebSockets.
6. **Automated Billing & Cashless Settlement:** Timestamp-based billing removes human calculation errors, while **GCash via PayMongo API v1** ensures webhook-verified digital payments alongside cash options."

CHAPTER 3: METHODOLOGY

SLIDE 7: Research Design and Development Method (~1 Minute)

Speaker:

"Beginning Chapter 3 on **Slide 7**, we utilized a **Descriptive-Developmental Research Design**. The **developmental** aspect covers the engineering of our multi-tenant cross-platform software, while the **descriptive** aspect quantifies software quality through **IT developer evaluation** under **ISO/IEC 25010:2011**.

We followed the **Incremental Development Model** across five sequential increments:

- **Increment 1:** User authentication, Laravel Sanctum role-based access control, KYC uploads, and **Tesseract OCR** vehicle plate registration.
- **Increment 2:** Parking space listings, admin fee boundary controls, and our interactive **2D floor plan slot configurator**.
- **Increment 3:** Geolocation search, **Google Maps** routing, and pessimistic row-locked reservation booking.
- **Increment 4: HMAC-SHA256 QR** entry and exit scanning via Google ML Kit, automated duration and overstay billing, and **GCash PayMongo** plus Cash checkout.
- **Increment 5:** Reservation-bound chat, **WebRTC** voice calling, **FCM and Brevo** notifications, dispute mediation, and Admin CSV forensic reporting."

SLIDE 8: System Architecture & Database Design (~1 Minute 15 Seconds)

Speaker:

"**Slide 8** illustrates our **Three-Tier System Architecture** and **Relational Database Design**.

- At the **Client Tier**, we have our **Flutter 3.x Android app** and **Laravel Blade + Tailwind CSS + Alpine.js web portal**.

- At the **Application Tier**, our **PHP 8.2 / Laravel 12 RESTful API** runs inside a **Docker and Nginx container on Render Cloud**, handling Sanctum authentication, OCR processing, HMAC signing, and billing logic.
- At the **Data Tier**, we use **Supabase Managed PostgreSQL 15+** with connection pooling, Row-Level Security, and Realtime Change Data Capture.

Our **Crow's Foot ERD** consists of **14 relational tables** organized into three functional subsystems:

1. The **Core Booking Subsystem (6 tables):** `users`, `vehicles`, `parking_spaces`, `parking_slots`, `reservations`, and `parking_space_feedback`.
2. The **Communications & Notification Subsystem (5 tables):** `reservation_messages`, `reservation_calls`, `reservation_call_signals` for WebRTC signaling, `notifications`, and `user_device_tokens`.
3. The **Security Audit & Telemetry Subsystem (3 tables):** `activity_logs`, `qr_transaction_logs`, and `support_requests`."

SLIDE 9: Proposed Prototype & User Interfaces (~1 Minute 15 Seconds)

Speaker:

"**Slide 9** showcases our actual cross-platform user interfaces across all three roles:

- **First, on the Driver Interface:** Motorists can browse verified Calamba City parking pins on **Google Maps**, filter by vehicle compatibility, and inspect a property's **interactive 2D floor plan** to pick a specific slot. In **My Reservations**, drivers track their booking across a **6-stage live progress bar**—from *Requested* to *Paid*—view their encrypted **Entry QR Code**, and message or call the provider via WebRTC.
- **Second, on the Parking Space Provider Interface:** The Owner Dashboard displays real-time counters for active slots, pending requests, and payments. Providers can visually lay out slots on a **2D grid** and use the built-in **Entry and Exit Checkpoint Scanner**—powered by Google ML Kit—or enter the **`RES-#` reference code** manually if a driver's screen is damaged.
- **Third, on the System Administrator Interface:** Accessible on both mobile and web, administrators perform side-by-side audits of KYC IDs, property documents, and OCR-extracted license plates, monitor live QR telemetry and reservations, resolve disputes, and export **CSV audit reports** with one click."

SLIDE 10: Hardware and Software Components (~45 Seconds)

Speaker:

"**Slide 10** summarizes our Hardware and Software Specifications.

- For **Hardware**, development was conducted on PCs with at least an Intel Core i3 or Ryzen 3 processor and 8 GB RAM, while field testing used **Android smartphones running Android 8.0 or higher** equipped with camera, GPS, and microphone sensors over Wi-Fi and 4G LTE.

- For **Software**, our updated **E-Parada (16) stack** pairs **Flutter 3.x** and **Laravel 12 (PHP 8.2)** on **Docker and Render** with **Supabase PostgreSQL 15+**. Key engines include our custom **2D Floor Plan Slot Engine**, **Google Maps SDK**, **Tesseract OCR CLI** with LTO regex validation, **Google ML Kit & CameraX (`mobile_scanner`)** with `qr_flutter`, **WebRTC (`flutter_webrtc`)**, **Firebase Cloud Messaging**, **Brevo SMTP**, and **PayMongo API v1** for GCash."

SLIDE 11: Data Flow and Control Logic (~1 Minute 30 Seconds)

Speaker:

"**Slide 11** details our system's Data Flow and Control Logic.

- Our **Cross-Functional Activity Diagram and 12-Phase Flowchart** map the complete lifecycle across five swimlanes—from KYC and OCR onboarding in Phases 1 to 3, to 2D slot listing and row-locked reservation in Phases 4 to 6, QR gate check-in/check-out and automated billing in Phases 7 to 9, and Admin governance in Phases 10 to 12.
- Our **Partitioned Data Flow Diagrams** decompose the system from the **Level-0 Context and Core Processes** across 6 data stores, down to **Level-1 (Driver Search & Reservation)** and **Level-2 (HMAC-SHA256 QR Pass Generation)**.
- Underneath, four core technical subroutines power the backend:
  1. **Subroutine 1 (Tesseract OCR):** Applies grayscale conversion, contrast boosting, and adaptive thresholding before running **PSM 11 sparse-text extraction** and Philippine LTO plate regex matching.
  2. **Subroutine 2 (Pessimistic Concurrency Locking):** Executes `SELECT ... FOR UPDATE` on the target `parking_slots` row inside an ACID transaction and checks for temporal overlap before committing, guaranteeing **zero double-bookings**.
  3. **Subroutine 3 (Cryptographic QR Replay Guard):** Recomputes the **256-bit HMAC-SHA256 signature**, verifies single-use state and checkpoint matching, and records an immutable entry in `qr_transaction_logs`.
  4. **Subroutine 4 (Automated Duration & Overstay Billing):** Calculates exact elapsed time (time_out − time_in), applies a **15-minute grace period**, computes ceiling-rounded base and overstay charges with zero platform commission deduction, and finalizes payment via Cash or **PayMongo GCash webhooks**."

SLIDE 12: Testing Environment, Phases, and Test Coverage (~1 Minute)

Speaker:

"On **Slide 12**, we validated E-Parada across physical Android smartphones (Android 8.0 to 14.0), web browsers, and our cloud-hosted Render and Supabase environment under Wi-Fi, 4G LTE, and throttled network conditions.

Testing followed **three sequential phases**:

1. **Alpha Testing:** Unit and modular verification across all 5 development increments.
2. **System & Scenario Evaluation:** End-to-end role-based workflow testing conducted by purposively selected **IT Developers**.
3. **Regression Testing:** Re-verifying all modules after every increment to ensure zero regression defects.

In total, our **20-item Test Case Suite (TC-01 to TC-20)** covers Sanctum RBAC security, Tesseract OCR accuracy, 2D slot rendering, Google Maps routing, **concurrent booking stress tests**, **QR replay-attack rejection**, automated overstay billing math, PayMongo webhook settlement, and WebRTC voice calling."

SLIDE 13: Evaluation and Statistical Treatment (~1 Minute)

🎙️ **Speaker:**

"Finally, on **Slide 13**, our system evaluation is conducted using **Purposive Sampling of qualified IT Developers and Software Engineering Professionals**.

Each IT developer executes complete Driver, Provider, and Administrator workflows, inspects our database concurrency and QR security controls, and evaluates the software using the **ISO/IEC 25010:2011 Software Product Quality Model** across four core characteristics:

- **Functional Suitability**
- **Performance Efficiency**
- **Usability**
- **Reliability**

Alongside the survey, we measure **six objective technical benchmarks**: REST API latency under 350 milliseconds, QR validation under 600 milliseconds, **zero double-bookings under 25 simultaneous requests**, OCR extraction under 1.8 seconds with at least 90% precision, WebRTC audio latency under 200 milliseconds, and PayMongo webhook turnaround under 1.2 seconds.

Survey responses are analyzed using **Percentage Distribution**, **Weighted Mean**, and **Standard Deviation**, and interpreted using a **5-Point Likert Scale** where **4.21 to 5.00** corresponds to **Excellent / Highly Compliant**.

That concludes our presentation of **E-Parada**. Thank you, and we are now ready for your questions and recommendations."